
LAPORAN PRAKTIKUM
MATA KULIAH TEKNOLOGI IOT II

MODUL 5: TOPOLOGI JARINGAN TERDESENTRALISASI
(ESP-NOW DAN MESH)



Oleh:

Nama: Aila Nadianti

NPM : 25782059

Kelas : 3C

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026

5.4 Persiapan Praktikum

5.4.1 Kebutuhan Perangkat Keras

Praktikum ini dirancang sebagai tugas kolaboratif kelompok. Setiap kelompok minimal terdiri dari 2 orang mahasiswa (idealnya 3 orang), dengan masing-masing memegang 1 unit NodeMCU.

Pastikan komponen berikut tersedia di meja kelompok Anda:

- *Board* NodeMCU ESP8266 (Minimal 2 unit, disarankan 3 unit)
- Kabel Micro-USB Data (1 per mahasiswa)
- *Breadboard* / *Project Board* (secukupnya)
- Sensor Suhu & Kelembapan DHT11 atau DHT22 (1 buah per board / mahasiswa)
- Sensor Cahaya LDR (1 buah)
- Resistor 10k Ohm (1 buah, pembagi tegangan LDR)
- LED 5mm (1 buah)
- Resistor 220 Ohm (1 buah, pembatas arus LED)
- Kabel *Jumper* (*Male-to-Male* dan *Male-to-Female* secukupnya)

5.4.2 Persiapan Perangkat Lunak & Pustaka

Sebelum menjalankan praktikum jaringan *Mesh*, Anda wajib memasang pustaka pihak ketiga melalui Library Manager:

1. Pada Arduino IDE, buka menu **Sketch > Include Library > Manage Libraries...**
2. Ketikkan kata kunci "**painlessMesh**" di kolom pencarian (oleh *Coopdis*, *Scotty*, *etc.*).
3. Klik tombol **Install**. Saat muncul jendela konfirmasi dependensi pustaka tambahan, pastikan Anda mengeklik "**Install All**" agar pustaka pendukung berikut terpasang otomatis:
 - ArduinoJson (oleh Benoit Blanchon)
 - TaskScheduler (oleh Anatoli Arkhipenko)
 - ESPAsyncTCP (oleh dvarrel / me-no-dev)

5.5 Praktikum

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

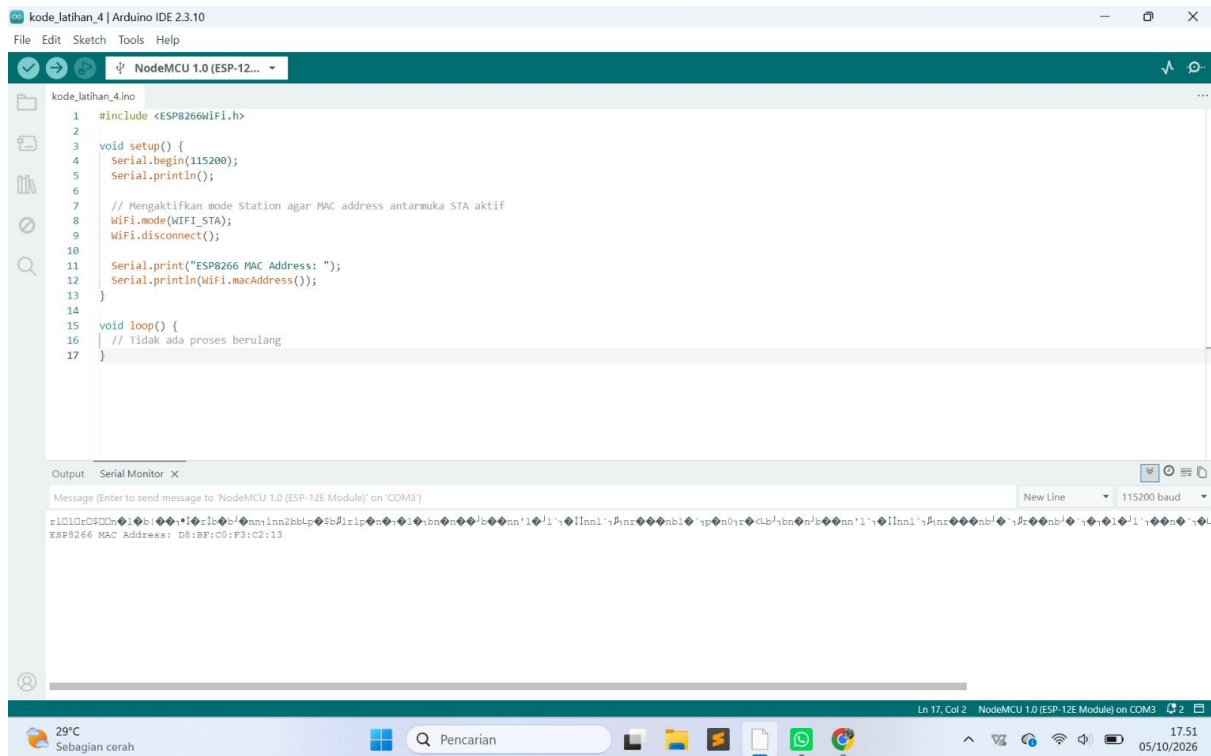
Sebelum membangun jaringan ESP-NOW, setiap anggota kelompok wajib membaca dan mencatat alamat fisik (*MAC Address*) dari masing-masing *board* NodeMCU.

MAC Syarifah sebagai Node1: EC:FA:BC:1D:BB:F8

MAC Sofy sebagai Node2: 34:94:54:64:9F:6C

MAC Aila sebagai Node3: DB:BF:CO:F3:02:13

Pada praktikum ini saya menjalankan program untuk mengetahui MAC Address NodeMCU. Program mengatur NodeMCU dalam mode Station, kemudian mengambil alamat MAC menggunakan WiFi.macAddress(). Hasilnya ditampilkan pada Serial Monitor saya, yaitu DB:BF:C0:F3:C2:13.



```
1 #include <ESP8266WiFi.h>
2
3 void setup() {
4   Serial.begin(115200);
5   Serial.println();
6
7   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8   WiFi.mode(WIFI_STA);
9   WiFi.disconnect();
10
11   Serial.print("ESP8266 MAC Address: ");
12   Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16   // Tidak ada proses berulang
17 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

ESP8266 MAC Address: DB:BF:C0:F3:C2:13

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

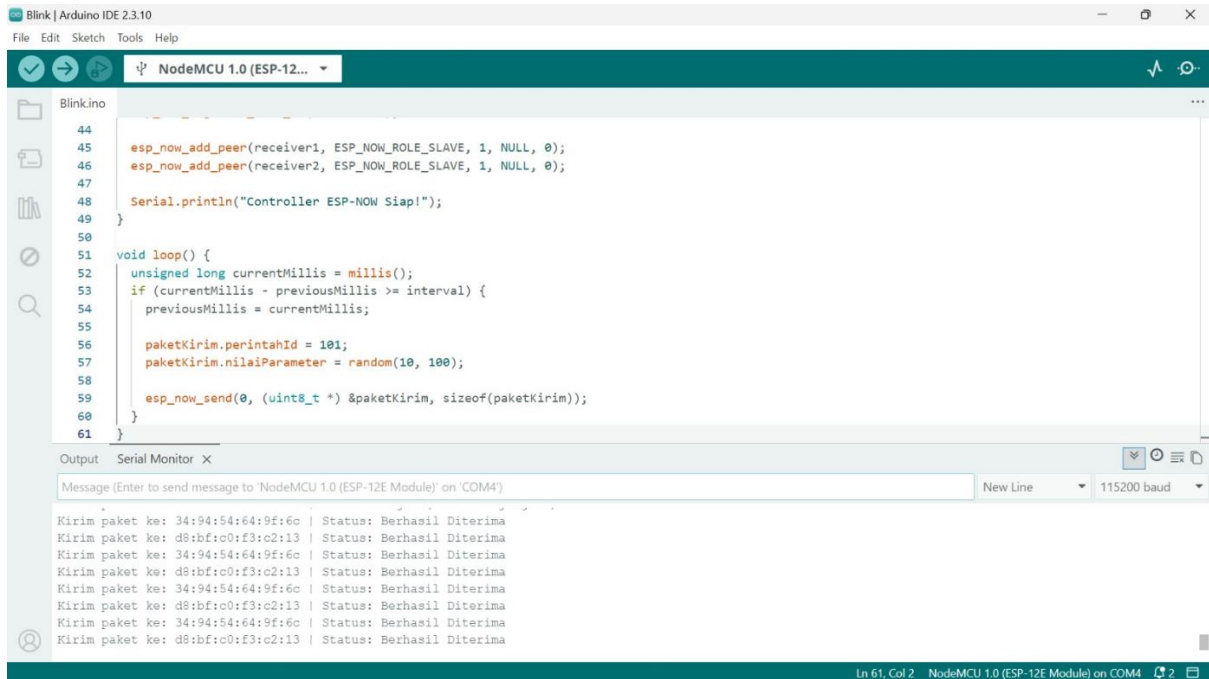
Skenario: Satu unit NodeMCU bertindak sebagai **Controller / Sender**, menyiarkan paket data instruksi yang sama secara langsung ke dua unit NodeMCU lain yang bertindak sebagai **Slave / Receiver** tanpa perantara router.

Langkah Kerja Pemrograman Sisi Pengirim (Controller):

1. Masukkan alamat MAC milik Board Penerima 1 dan Board Penerima 2 ke dalam variabel receiver1 dan receiver2.
2. Format tanda titik dua (:) diubah menjadi bilangan heksadesimal berawalan 0x yang dipisahkan oleh tanda koma (contoh: BC:FF:4D:23:45:90 → {0xBC, 0xFF, 0x4D, 0x23, 0x45, 0x90}).

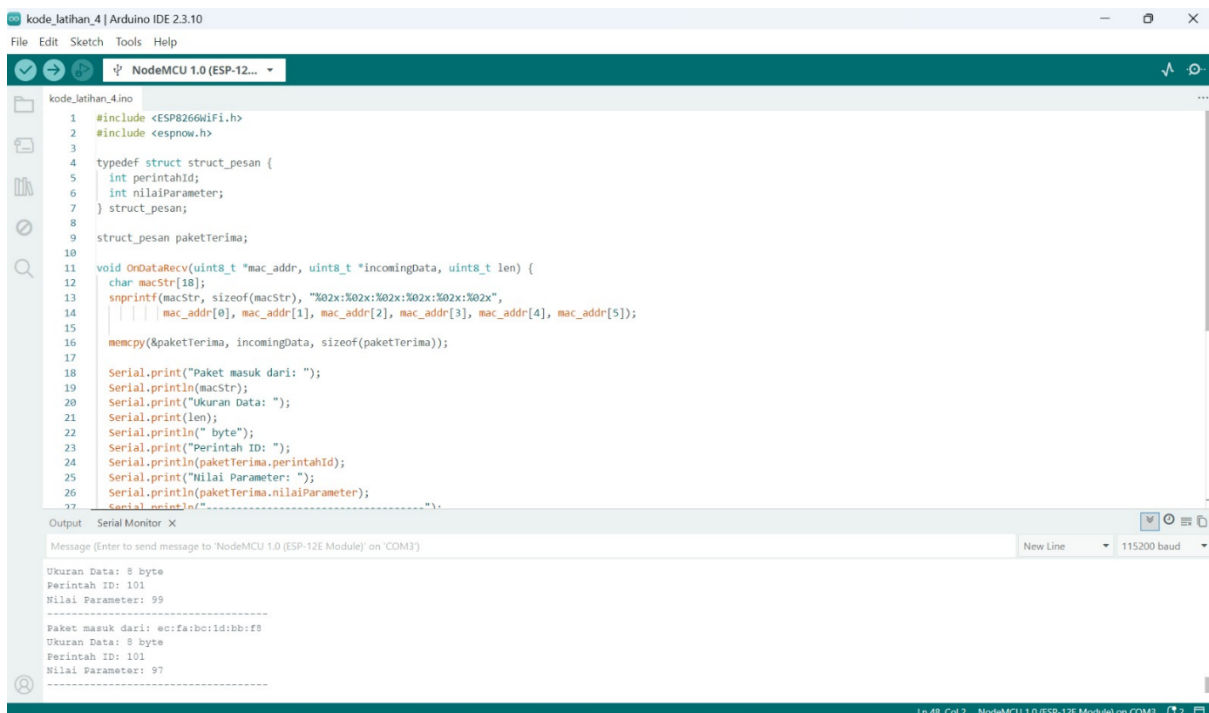
Sisi Pengirim (Controller):

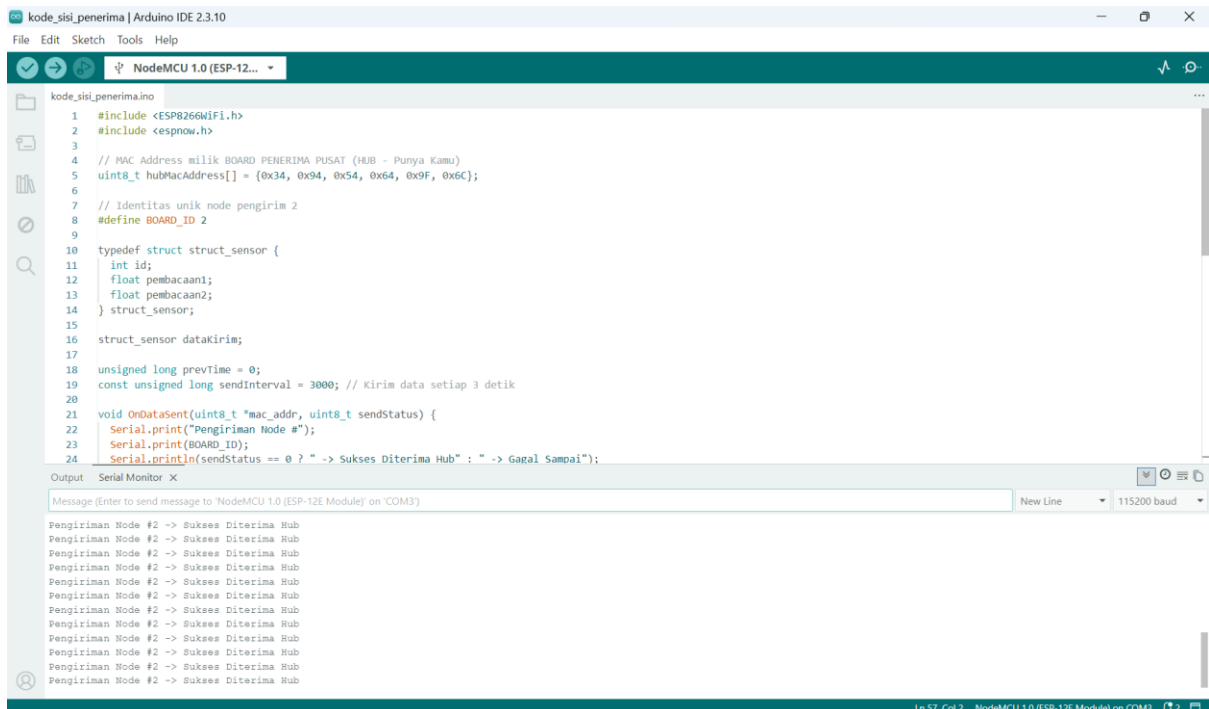
Pada praktikum ini NodeMCU berperan sebagai Controller atau pengirim. Controller mengirim paket data ke dua NodeMCU penerima menggunakan ESP-NOW. Data yang dikirim memiliki ID 101 dan nilai parameter yang dibuat secara acak. Dari Serial Monitor terlihat status berhasil diterima, jadi komunikasi ESP-NOW antara pengirim dan penerima berhasil.



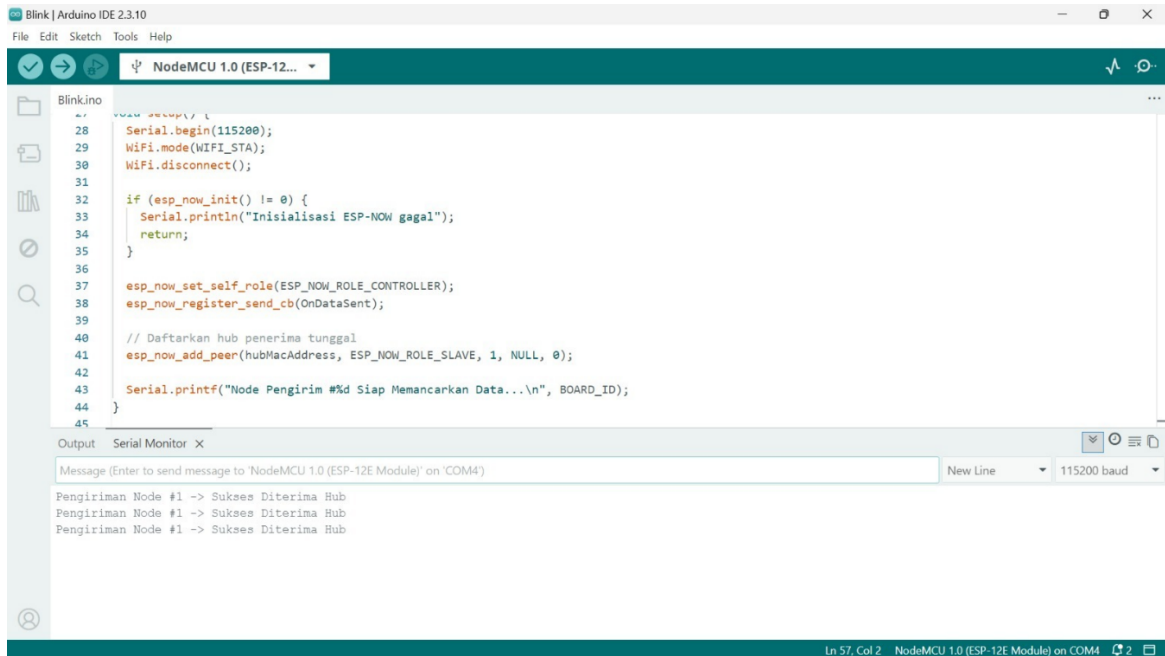
Sisi Penerima (Slave):

Pada bagian ini NodeMCU berfungsi sebagai penerima atau Slave. Ketika Controller mengirim data menggunakan ESP-NOW, fungsi OnDataRecv akan menerima data tersebut dan menampilkannya di Serial Monitor. Dari hasil percobaan terlihat data sebesar 8 byte berhasil diterima dengan Perintah ID 101 dan nilai parameter 99, kemudian 97.





Pada praktikum ini NodeMCU digunakan sebagai pengirim dengan BOARD_ID 2. Data sensor dikirim menuju satu NodeMCU pusat atau Hub berdasarkan MAC Address Hub. Pengiriman dilakukan setiap 3 detik. Dari Serial Monitor terlihat pengiriman dari Node 2 berhasil diterima oleh Hub.



The screenshot shows the Arduino IDE interface with the 'Blinkino' sketch loaded. The code is for a NodeMCU 1.0 (ESP-12E Module) configured as a controller. It initializes the serial port at 115200 baud, sets the role to controller, and registers a callback for data sent. The serial monitor shows three successful transmissions from Node #1 to the Hub.

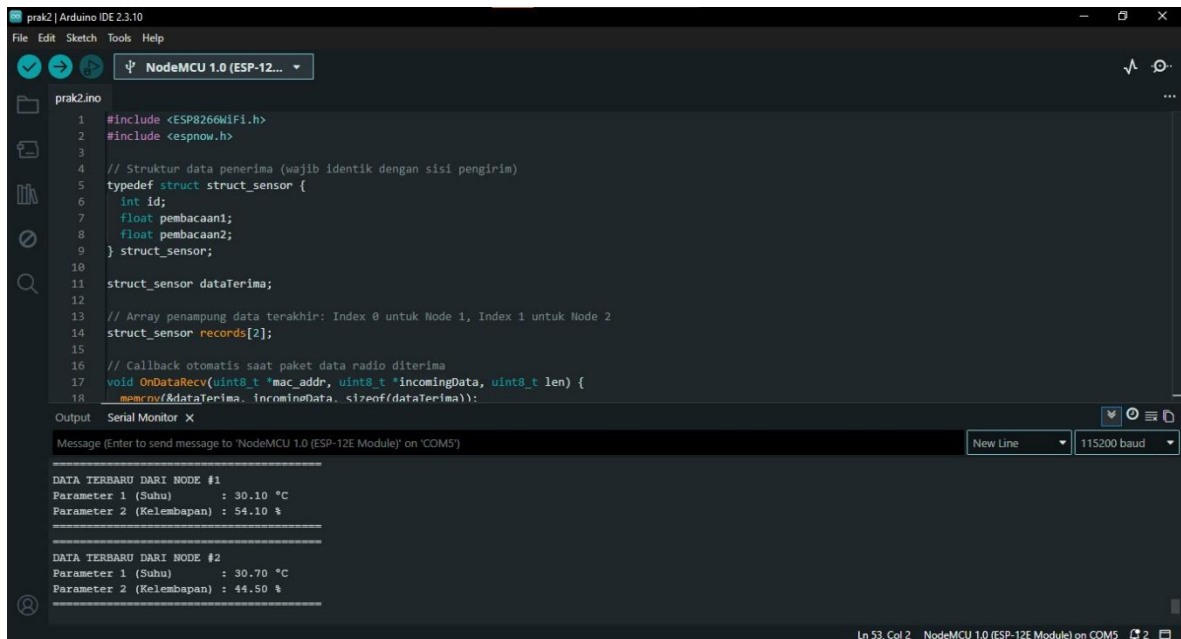
```
27 void setup() {
28   Serial.begin(115200);
29   WiFi.mode(WIFI_STA);
30   WiFi.disconnect();
31
32   if (esp_now_init() != 0) {
33     Serial.println("Inisialisasi ESP-NOW gagal");
34     return;
35   }
36
37   esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
38   esp_now_register_send_cb(OnDataSent);
39
40   // Daftarkan hub penerima tunggal
41   esp_now_add_peer(hubMacAddress, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
42
43   Serial.printf("Node Pengirim #%d Siap Memancarkan Data...\n", BOARD_ID);
44 }
45
```

Serial Monitor Output:

```
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
```

Sisi Hub Konsentrator:

Pada bagian ini NodeMCU berfungsi sebagai Hub atau pusat penerima. Hub menerima data dari Node 1 dan Node 2, kemudian menyimpan dan menampilkan data suhu serta kelembapan. Dari hasil Serial Monitor, Node 1 mengirim suhu 30,10°C dan kelembapan 54,10%, sedangkan Node 2 mengirim suhu 30,70°C dan kelembapan 44,50%. Berarti komunikasi Many-to-One berhasil.



The screenshot shows the Arduino IDE interface with the 'prak2.ino' sketch loaded. The code is for a NodeMCU 1.0 (ESP-12E Module) configured as a slave. It includes the ESP8266WiFi.h and espnow.h libraries. It defines a struct for sensor data (temperature and humidity) and an array to store data from two nodes. The serial monitor shows data received from Node #1 and Node #2.

```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct struct_sensor {
6   int id;
7   float pembacaan1;
8   float pembacaan2;
9 } struct_sensor;
10
11 struct_sensor dataTerima;
12
13 // Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
14 struct_sensor records[2];
15
16 // Callback otomatis saat paket data radio diterima
17 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
18   memcpy(&dataTerima, incomingData, sizeof(dataTerima));
19 }
```

Serial Monitor Output:

```
DATA TERBARU DARI NODE #1
Parameter 1 (Suhu)      : 30.10 °C
Parameter 2 (Kelembapan) : 54.10 %

DATA TERBARU DARI NODE #2
Parameter 1 (Suhu)      : 30.70 °C
Parameter 2 (Kelembapan) : 44.50 %
```

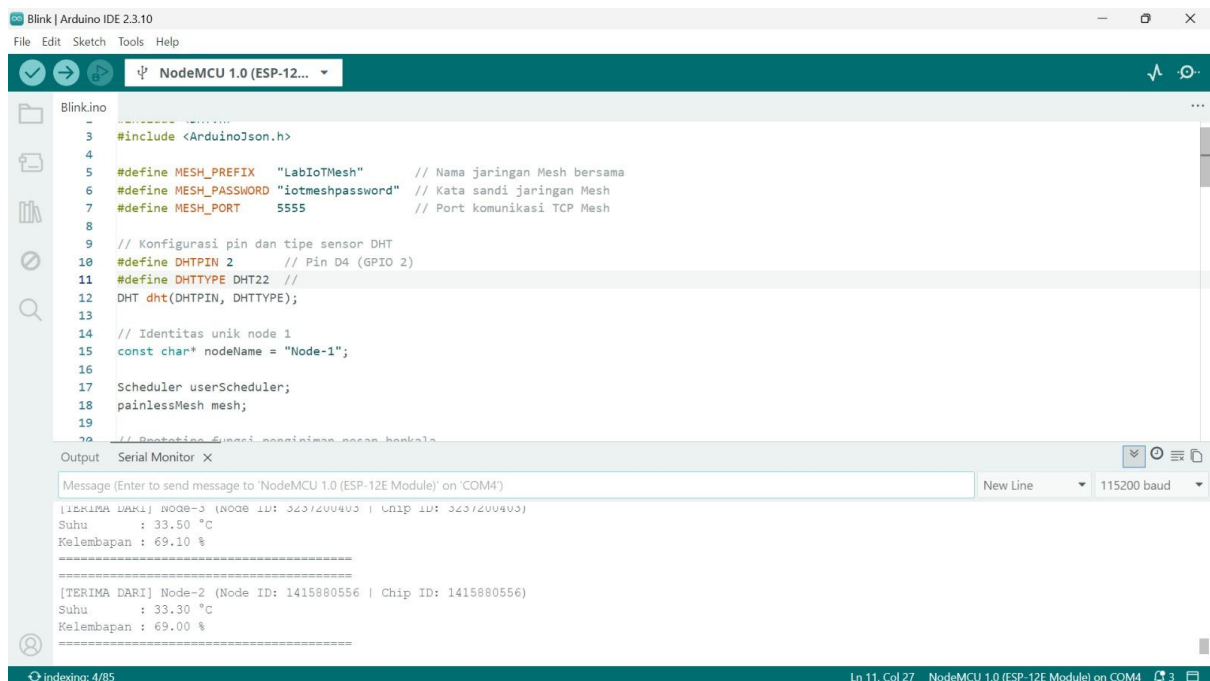
5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

Skenario: Seluruh board dalam kelompok dihubungkan ke sensor suhu & kelembapan riil (DHT11/DHT22) dan diintegrasikan ke dalam sebuah jaringan rantai otonom (*Mesh*). Setiap board secara berkala membaca sensor fisiknya, mengemas data telemetri ke dalam format **JSON**, dan menyiarkannya ke seluruh anggota jaringan tanpa perlu mengetahui alamat MAC masing-masing.

Instruksi Perangkaian Perangkat Keras:

- Pasang sensor **DHT11 / DHT22** pada masing-masing board NodeMCU:
 - Pin **VCC** sensor ke pin **3V3 (3.3V)** NodeMCU.
 - Pin **GND** sensor ke pin **GND** NodeMCU.
 - Pin **DATA / OUT** sensor ke pin **D4 (GPIO 2)** NodeMCU.

Pada praktikum ini kami menggunakan jaringan Mesh dengan painlessMesh. Node-1 membaca data dari sensor DHT22 dan juga menerima data dari node lain. Dari Serial Monitor terlihat Node-1 berhasil menerima data suhu dan kelembapan dari Node-2 dan Node-3.



The screenshot shows the Arduino IDE interface. The top toolbar includes icons for running, uploading, and monitoring. The main editor displays the following code for Node-1:

```
1 // Node-1
2
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "iotmeshpassword" // Kata sandi jaringan Mesh
7 #define MESH_PORT 5555 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 //
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node 1
15 const char* nodeName = "Node-1";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
20 // Restoring fungsi komunikasi secara berkala
```

The Serial Monitor at the bottom shows the following output:

```
[TERIMA DARI] Node-3 (Node ID: 3231200403 | Chip ID: 3231200403)
Suhu : 33.50 °C
Kelembapan : 69.10 %
=====
[TERIMA DARI] Node-2 (Node ID: 1415880556 | Chip ID: 1415880556)
Suhu : 33.30 °C
Kelembapan : 69.00 %
=====
```

The status bar at the bottom indicates the file is indexed (4/85) and the code is running on a NodeMCU 1.0 (ESP-12E Module) on COM4 at 115200 baud.

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "iotmeshpassword" // Kata sandi jaringan Mesh
7 #define MESH_PORT 5555 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GP10 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node 3
15 const char* nodeName = "Node-3";
16
17 Scheduler usersScheduler;
18 painlessMesh mesh;
19
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

```
[TERIMA DARI] Node-1 (Node ID: 3156065272 | Chip ID: 3156065272)
Suhu : 33.60 °C
Kelembapan : 67.80 %
=====
[RIRIM MESH] {"node":"Node-3","chipId":3237200403,"suhu":33.5,"kelembapan":69.1}
=====
[TERIMA DARI] Node-2 (Node ID: 1415880556 | Chip ID: 1415880556)
Suhu : 33.30 °C
Kelembapan : 68.90 %
=====
[TERIMA DARI] Node-1 (Node ID: 3156065272 | Chip ID: 3156065272)
Suhu : 33.60 °C
Kelembapan : 67.90 %
=====
[RIRIM MESH] {"node":"Node-3","chipId":3237200403,"suhu":33.5,"kelembapan":69}
```

Pada percobaan ini kami menggunakan painlessMesh untuk membuat jaringan Mesh. Setiap NodeMCU membaca suhu dan kelembapan dari DHT22, kemudian datanya dikemas dalam format JSON dan dikirim melalui jaringan Mesh. Pada Serial Monitor terlihat Node-1 menerima data dari Node-2 dan Node-3, jadi komunikasi antar-node berhasil.

LIBRARY MANAGER

sketch_oct6a.ino

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh" // Nama jaringan Mesh dengan teman
6 #define MESH_PASSWORD "iotmeshpassword" // Kata sandi jaringan Mesh
7 #define MESH_PORT 5555 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2
11 #define DHTTYPE DHT22
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

New Line 115200 baud

```
[TERIMA DARI] Node-3 (Node ID: 3237200403 | Chip ID: 3237200403)
Suhu : 33.50 °C
Kelembapan : 69.10 %
=====
[RIRIM MESH] {"node":"Node-2","chipId":1415880556,"suhu":33.3,"kelembapan":69}
=====
[TERIMA DARI] Node-1 (Node ID: 3156065272 | Chip ID: 3156065272)
Suhu : 33.70 °C
Kelembapan : 68.00 %
=====
[TERIMA DARI] Node-3 (Node ID: 3237200403 | Chip ID: 3237200403)
Suhu : 33.50 °C
Kelembapan : 69.10 %
=====
[RIRIM MESH] {"node":"Node-2","chipId":1415880556,"suhu":33.4,"kelembapan":69.1}
```

5.6 Latihan

Kerjakan latihan berbasis eksperimen, observasi serial monitor, dan modifikasi kode berikut untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:

- i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
- ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?

Jawab: Ya, status pengiriman untuk MAC address board yang dicabut akan berganti menjadi "Gagal (Tidak Terjangkau)".

- iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Jawab: ESP-NOW bisa mendeteksi kegagalan karena memanfaatkan fitur bawaan MAC ACK (sinyal konfirmasi balasan) di level *hardware* Wi-Fi (Layer 2 MAC). Kalau board yang dicabut mati, chip radionya otomatis nggak bisa ngirim balik sinyal ACK, sehingga pengirim langsung tahu kalau perangkat tersebut gagal dijangkau tanpa perlu pakai protokol TCP yang rumit.

- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):** Jalankan jaringan *painlessMesh* pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:

- i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?

Jawab: 3 hingga 10 detik.

- ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

Jawab: 5 hingga 15 detik.

```
1 // Blinkino
2
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "iotmeshpassword" // Kata sandi jaringan Mesh
7 #define MESH_PORT 5555 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 //
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node 1
15 const char* nodeName = "Node-1";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
20 // Definisi fungsi komunikasi mesh ke-kala
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

New Line 115200 baud

```
mesn node [node-1] berjalan. menunggu pementuan topologi...
[KIRIM MESH] {"node":"Node-1","chipId":3156065272,"suhu":33.8,"kelembapan":68}
--> Koneksi Baru Terdeteksi! Node ID: 1415880556
--> Topologi rantai mesh telah diperbarui
=====
[TERIMA DARI] Node-2 (Node ID: 1415880556 | Chip ID: 1415880556)
Suhu : 33.40 °C
Kelembapan : 69.10 %
=====
```

Ln 11, Col 27 NodeMCU 1.0 (ESP-12E Module) on COM4

- **Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.
 - i. Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
 - ii. Jika suhu bernilai normal ($\leq 32.0\text{ }^{\circ}\text{C}$), program cukup mencetak status singkat satu baris: `[NORMAL] Telemetri dari Node <chipId> aman`.
 - iii. Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

Sisi Pengirim (Controller):

Di percobaan ini NodeMCU membaca suhu dan kelembapan menggunakan DHT22. Data dikirim antar-node menggunakan jaringan Mesh dalam format JSON. Program juga memiliki kondisi peringatan suhu kritis, sehingga ketika suhu melewati batas yang ditentukan, muncul tulisan peringatan pada Serial Monitor `[PERINGATAN SUHU KRITIS!]`. Artinya program memiliki kondisi peringatan ketika suhu dianggap sudah melewati batas yang ditentukan dalam program.

```
sketch_oct6b.ino
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "iotmeshpassword" // Kata sandi jaringan Mesh
7 #define MESH_PORT 5555 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node kamu
15 const char* nodeName = "Node-2";
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

[PERINGATAN SUHU KRITIS!]
[TERIMA DARI] Node-3 (Node ID: 3237200403 | Chip ID: 3237200403)
Suhu : 33.30 °C
Kelembapan : 68.00 %

[PERINGATAN SUHU KRITIS!]
[TERIMA DARI] Node-1 (Node ID: 3156065272 | Chip ID: 3156065272)
Suhu : 33.60 °C
Kelembapan : 66.90 %

[KIRIM MESH] {"node":"Node-2","chipId":1415880556,"suhu":33.9,"kelembapan":68}

Ln 11, Col 23 NodeMCU 1.0 (ESP-12E Module) on COM5

Sisi Penerima (Slave):

```
5.5.4_Praktikum_4.ino
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "iotmeshpassword" // Kata sandi jaringan Mesh
7 #define MESH_PORT 5555 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node 3
15 const char* nodeName = "Node-3";
16
17 Scheduler usersScheduler;
18 painlessMesh mesh;
19
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

[TERIMA DARI] Node-2 (Node ID: 1415880556 | Chip ID: 1415880556)
Suhu : 34.10 °C
Kelembapan : 66.60 %

[KIRIM MESH] {"node":"Node-3","chipId":3237200403,"suhu":32.9,"kelembapan":68.6}

[TERIMA DARI] Node-1 (Node ID: 3156065272 | Chip ID: 3156065272)
Suhu : 33.50 °C
Kelembapan : 69.00 %

[TERIMA DARI] Node-2 (Node ID: 1415880556 | Chip ID: 1415880556)
Suhu : 34.10 °C
Kelembapan : 66.60 %

Ln 11, Col 22 NodeMCU 1.0 (ESP-12E Module) on COM3